



Programmation Orientée Objets

(II 1)

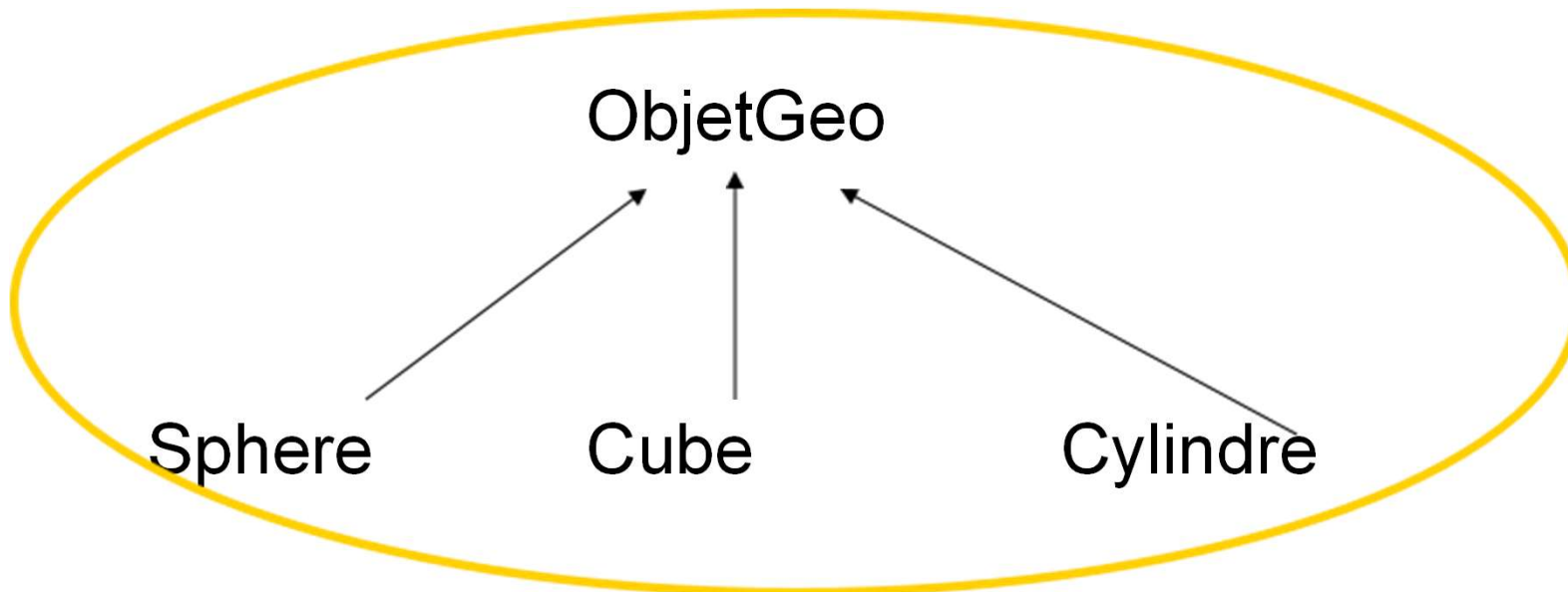
Le polymorphisme en C++



Polymorphisme

- **Introduction**
- **Conversion d'objets et trans-typage**
- **Fonction virtuelle**
- **Fonction virtuelle pure**
- **Classe abstraite**
- **Structure Polymorphe**
- **Destructeur virtuel**
- **Exemples**

Exemple d'héritage



- ObjetGeo: Super-classe ou classe de base. Classe très générale.
- Sphere, Cube, Cylindre: classe dérivée. Classes plus spécifiques.

Conversion d'objets

- Un objet de la classe dérivée peut-être converti implicitement en un objet de la classe de base.

```
class A
{ protected :int i;
public:
int f(){return(2*i);}
...};
```

```
class B : public A
{ int j;
public:
int f(){return(i*j);}
...};
```

```
ObjetGeo UnObjet;
Cube MonCube;

UnObjet = MonCube;
```

```
A a;
a.f();
```

```
B b;
b.f();
```

```
a = b;
a.f();
```

2

3

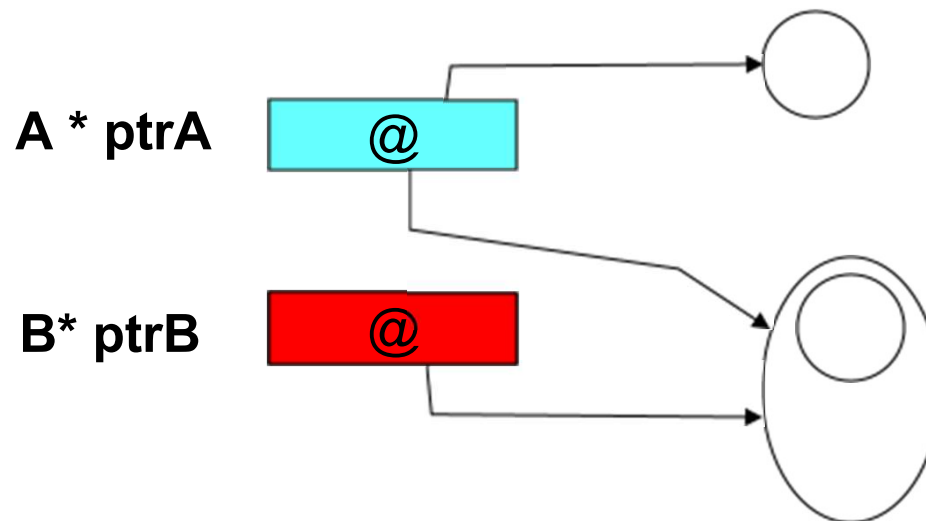
4

Conversion d'objets (suite)

- Un pointeur (ou une référence) sur un objet de la classe dérivée peut-être converti implicitement en un pointeur de la classe de base. (**upcasting**)

```
ObjetGeo * PtrObjet;  
PtrObjet = new Cube;
```

```
Cube monCube;  
ObjetGeo & obj = monCube;
```



Conversion d'objet (suite)

- Une conversion de la classe de base vers la classe dérivée n'est pas autorisée.

- **ObjetGeo MonObjet;**
- **Cube MonCube;**
- ~~**MonCube = MonObjet;**~~
- **En cas de besoin, il faut créer la surcharge de l'opérateur `=(const ObjetGeo &)` dans la classe MonCube qui copie la partie commune et initialise la partie spécifique**

Que se passe t-il ?

- Un pointeur d'une classe de base pointe ou référence une instance dynamique de la classe dérivée. (*PtrObjet pointe vers un espace mémoire de type Cube*)
 - **Polymorphisme** = plusieurs formes
- A partir du pointeur PtrObjet, on ne peut accéder qu'à l'interface public de la classe ObjetGeo.
- Dans le cas où la classe dérivée redéfinit les méthodes de la classe de base, doit-on invoquer les méthodes de la classe dérivée ou de la classe de base.

deux cas : sans virtualité ou avec virtualité



Fonction virtuelle

- La fonction virtuelle est utilisée afin de permettre l'utilisation d'objets de façon appropriée dans un programme où le type des objets n'est pas connu à la compilation.
- Le mot clé **virtual** indique que la fonction peut avoir différentes versions pour les différentes classes dérivées.



- Lors de l'exécution du programme, la bonne version de fonction est exécutée dynamiquement.

■ **Liaison (ligature) dynamique (virtualité)**

La classe ObjetGeo

```
class ObjetGeo
{
    Position P;
public:
    ObjetGeo();
    virtual void afficher();
    ...
};
```

Fonction virtuelle
afficher()



```
void ObjetGeo::afficher()
{
    cout << "Position " << P << endl;
}
```

La classe Sphere dérivée de la classe ObjetGeo

```
class Sphere : public ObjetGeo
{
public:
    Sphere(int r);
    void afficher();
private:
    float rayon;
};
```

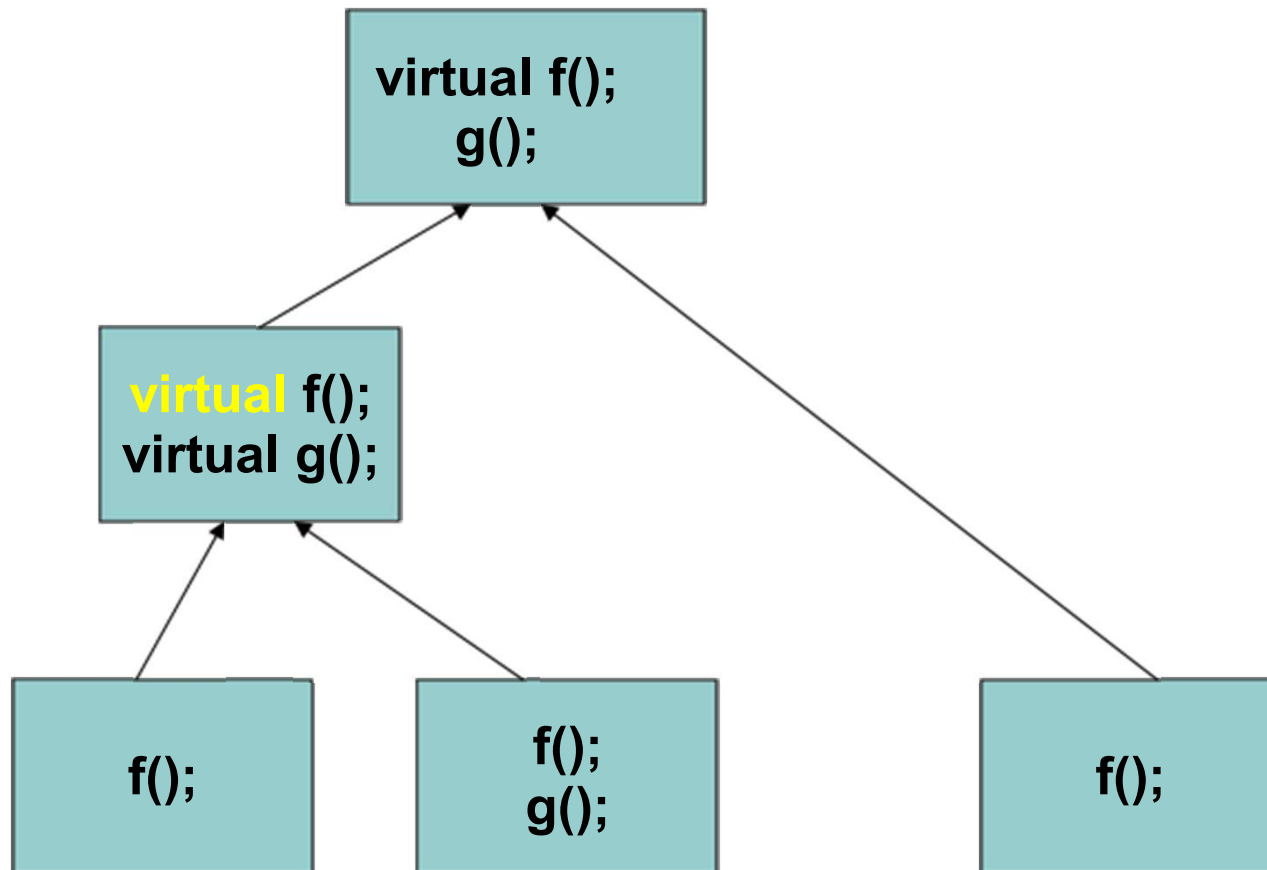
Redéfinition de la méthode afficher() dans la classe Sphere.



```
ObjetGeo *ptr;
ptr=new Sphere(7);
ptr->afficher();
```

```
void Sphere::afficher()
{
    ObjetGeo::afficher();
    cout << "Rayon: " << rayon << endl;
}
```

Portée de la virtualité



Destructeur virtuel

- Un destructeur peut être défini comme virtuel.
Cependant, un constructeur ne pourra jamais l'être.

■ Contexte

- Destruction d'un objet de la classe dérivée qui est référencé par un pointeur de la classe de base,

■ Résultat

- Si le destructeur est virtuel
alors le destructeur de la classe dérivée est appelé
.
Sinon seul le destructeur de la classe de base est appelé.

Destructeur virtuel (exemple)

ObjetGeo *OB = new Cube();

```
class ObjetGeo {  
public:  
    virtual ~ ObjetGeo();  
};  
  
class Cube {  
public:  
    ~ Cube();  
};
```

```
class ObjetGeo {  
public:  
    ~ ObjetGeo();  
};  
  
class Cube {  
public:  
    ~ Cube();  
};
```

delete OB;

Le destructeur de **Cube** est appelé en premier
+ le destructeur de **ObjetGeo** sera appelé.

seul le destructeur de
ObjetGeo sera appelé.

Fonction virtuelle pure et classe abstraite

- Certaines classes représentent des concepts abstraits par lesquels des objets ne peuvent jamais exister.
- **Exemple:** Un objet géométrique ne peut exister sans savoir comment sa forme est représentée.
- On déclare des **fonctions virtuelles pures par un initialisateur = 0**.
- Une classe qui possède une fonction virtuelle pure est **une classe abstraite** et **ne peut donc jamais être instanciée**
- Les classes dérivées implémentent à leur niveau ces fonctions.

Fonction virtuelle pure

```
class ObjetGeo
{ ...
public:
...
virtual float volume() = 0;
protected
Position P;
};
```

```
class Cube : public ObjetGeo
{ float Larg,Long,Haut;
public:
float volume(){
return (Larg * Long *Haut);
}
};
```

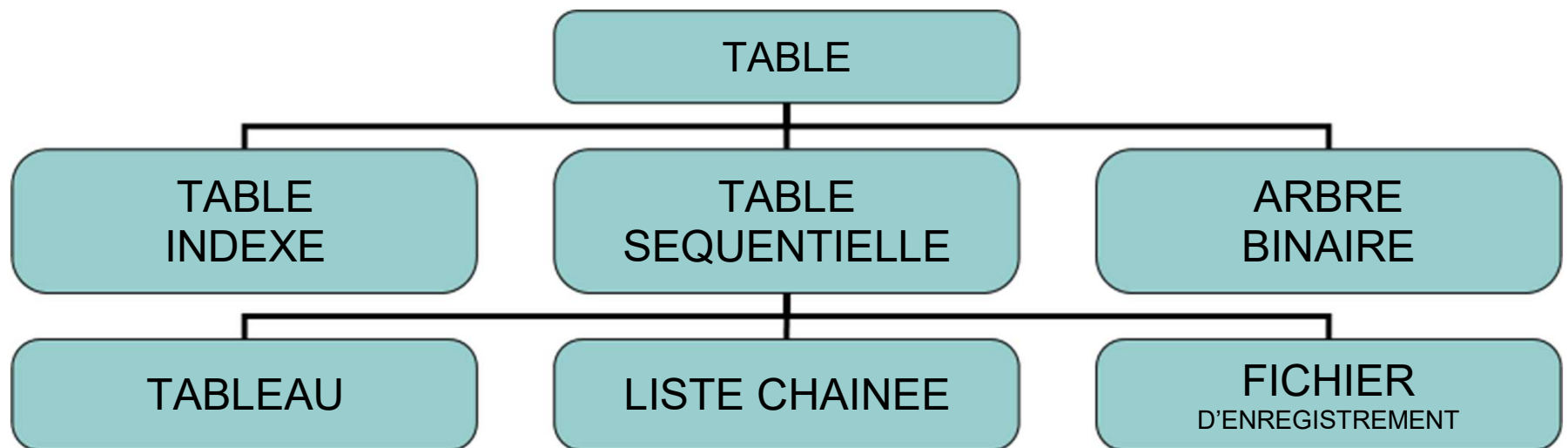
```
class Sphere : public ObjetGeo
{ int rayon;
public:
float volume(){
return ((4/3)*rayon^3*PI);
}
};
```

Classe abstraite

- Une **classe abstraite** sert pour factoriser. Aussi, elle joue le rôle d'une **interface** commune à plusieurs implémentations.
- La **classe abstraite ne peut pas** être passée en paramètre comme objet statique.
- La **classe abstraite peut cependant être passée par pointeur ou par référence** (*polymorphisme*).
- **Attention !** Les sous-classes d'une classe abstraite **doivent implémenter** ces fonctions virtuelles pures, **sinon** elles aussi deviennent abstraites.

Exemple : Recherche dans une table

Variantes d'implémentation => classe abstraite + des classes dérivées



Recherche séquentielle : variantes d'implémentation

	<i>Tableau</i>	<i>Liste Chaînée</i>	<i>Fichier Séquentiel</i>
<i>Début de la recherche : Position_initiale</i>	$i = 1$	$l = \text{tête}$	<i>ouvrir Lecture</i>
<i>Avancer à la position suivante : Avancer</i>	$i = i + 1$	$l = l.\text{suivant}$	<i>lire(enreg)</i>
<i>Comparaison : Trouvé</i>	$t[i] == x$	$l.\text{valeur} == x$	$\text{Enreg} == X$
<i>Test de fin de parcours : Epuisé</i>	$i > \text{taille}$	$l == \text{null}$	<i>Eof</i>

Exemple (suite)

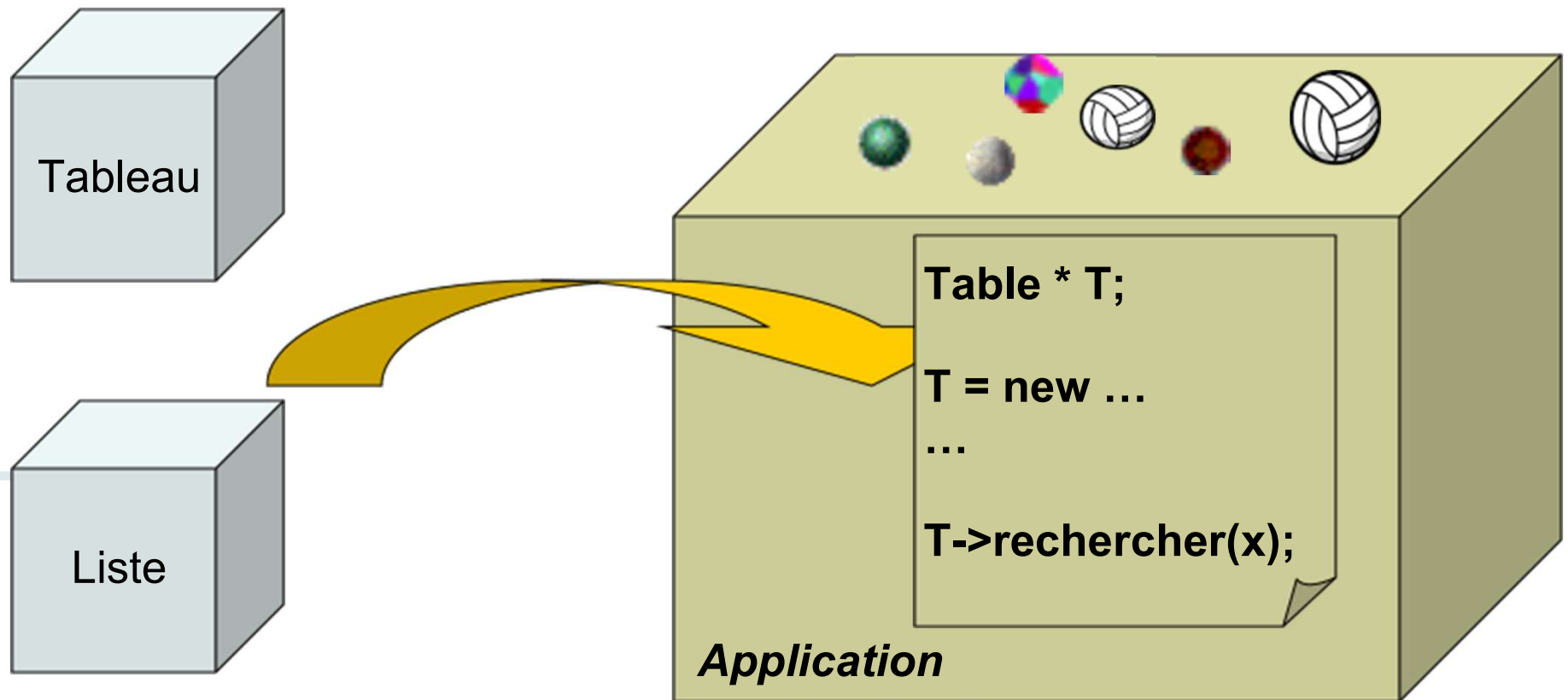
```
class Table
{
public:
    bool rechercher(X x);
    virtual void demarrer() = 0;
    virtual void avancer() = 0;
    virtual bool trouve() = 0;
    virtual bool fin() = 0;
};
```

```
bool Table::rechercher(X x)
{
    début
    demarrer();
    while (!fin() && !trouve(x))
        avancer();
    return ( !fin() );
};
```

```
class Tableau : public Table
{ X t[MAX];
public:
    void demarrer() ;
    void avancer();
    bool trouve();
    bool fin() ;
};
```

```
class Liste : public Table
{ struct element {
    X valeur;
    element * suivant;
} * tete;
public:
    // demarrer(); avancer(); trouve(); fin();
};
```

Exemple (suite)



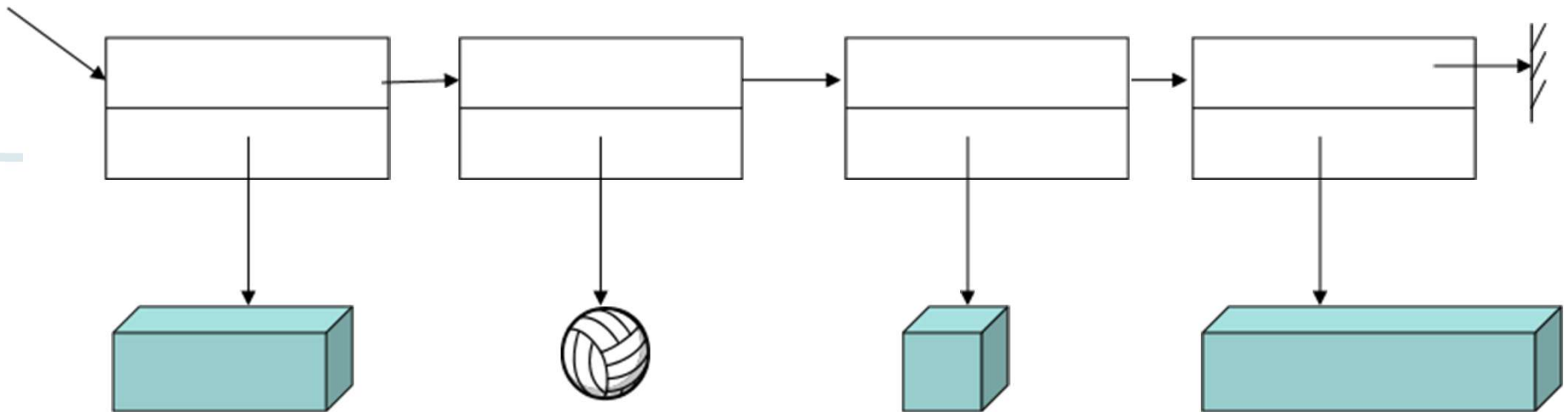
Structure Polymorphe =

Collection d'objets d'une classe de base comportant des méthodes virtuelles

Soit `ObjetGeo *Tab[N];`

Tab est rempli d'objets de type Sphere et/ou Cube

```
float Somme=0;  
for(i=0; i<N; i++)  
Somme+=Tab[i]->volume();
```



L'opérateur typeid

- l'opérateur *typeid* permet de connaître le type d'un objet lors de l'exécution
- On parle de **RTTI – Run-Time Type Information**

```
ObjetGeo *Tab[N];  
  
int nbCube=0;  
for(i=0; i<N; i++)  
if ( typeid( Tab[i] ) == typeid( Cube))  
nbCube++;
```

Attention: il faut éviter le plus possible d'utiliser cet opérateur

Mieux...

```
class ObjetGeo {
public:
virtual bool estCube() { return false; }
};

class Cube : public ObjetGeo {
public:
bool estCube() { return true; }
};

Int main (){
ObjetGeo *Tab[N];
int nbCube=0;
for(i=0; i<N; i++)
if (Tab[i]->estCube())
nbCube++;
}
```